

Final Studies Project Report

Presented in order to obtain

National Diploma of Engineering in Applied Science Technology
speciality : Embedded Systems and Connected Objects

By

Walid Ben Abi Taieb

Comparative assessment of hardware acceleration on the Edge AI

Professional supervisor: Mr. Enrico Mezzetti
Academic supervisor: Mrs. Mariem Feki

Host organisation: Barcelona supercomputing center



Final Studies Project Report

Presented in order to obtain

National Diploma of Engineering in Applied Science Technology
speciality : Embedded Systems and Connected Objects

By

Walid Ben Abi Taieb

Comparative assessment of hardware acceleration on the Edge AI

Professional supervisor : Mr. Enrico Mezzetti

Academic supervisor: Mrs. Mariem Feki

Host organisation: Barcelona supercomputing center



I authorize the student to submit their internship report for validation.

Professional supervisor , **Mr. Enrico Mezzetti**

Signature and stamp

I authorize the student to submit their internship report for validation.

Academic supervisor , **Mrs. Mariem Feki**

Signature

Dedications

Walid Ben Abi Taieb

Acknowledgements

Contents

General introduction	1
1 General context of the project	2
1.1 Host organisation	3
1.1.1 Barcelona super computing center	3
1.1.2 High Performance and Embedded Systems Laboratory (HPES)	3
1.2 Problem statement	4
1.2.1 The computational bottleneck of Edge AI	4
1.2.2 Hardware acceleration as the key enabler	5
1.2.3 The configuration challenge	5
1.2.4 Relevance to this work	5
1.3 Solution approach	6
1.4 Presentation of the proposed solution	6
1.5 Work process and documentation	7
2 Requirements Analysis and Operating Environment	8
2.1 Requirements Analysis	9
2.1.1 Functional Requirements	9
2.1.2 Non-Functional Requirements	10
2.2 System Architecture	11
2.2.1 Host PC	12
2.2.2 Target Board NXP i.MX 8M Plus	12
2.2.3 QEMU Emulation Layer	13
2.2.4 Data Flow	13
2.3 System Modeling	13
2.4 Work Environment	13
2.4.1 Hardware Environment	13
2.4.2 Software Environment	14

3	Emulation and Simulation Framework	17
3.1	Shared Pipeline Infrastructure	18
3.1.1	Four-Stage Pipeline Structure	18
3.1.2	Model Zoo	19
3.1.3	Dataset Construction	19
3.2	Methodology 1: x86 CPU Baseline	21
3.2.1	Overview	21
3.2.2	ONNX Runtime Session Configuration	21
3.2.3	Producer–Consumer Pipeline	21
3.2.4	Timing Isolation	22
3.2.5	Warm-up Protocol	22
3.2.6	Inference Algorithm	23
3.2.7	Statistical Metrics	23
3.2.8	Experimental Setup	24
3.2.9	Results	24
3.2.10	Latency Distribution: Histogram and KDE Visualisation	25
3.3	Methodology 2: Physics-Based ARM and NPU Simulation	29
3.3.1	Overview and Motivation	29
3.3.2	Target Hardware Specification	29
3.3.3	Tri-Path Architecture and Two-Pass Strategy	30
3.3.4	Path B: ARM Cortex-A53 CPU Simulator	30
3.3.5	Path C: Vivante GC7000UltraLite NPU Simulator	32
3.3.6	Per-Model Parameter Justification	33
3.3.7	Simulation Results	33
3.3.8	Visualisation Suite	34
3.3.9	Latency Distribution: Histogram and KDE Visualisation	34
3.3.10	Validity Scope and Limitations	37
3.4	Methodology 3: QEMU User-Mode ARM64 Emulation	37
3.4.1	Overview and Motivation	37
3.4.2	Two-Stage Docker Image Build	37
3.4.3	Pipeline Stages	38

3.4.4	Observed Failure and Root-Cause Analysis	38
3.4.5	Failure interpretation	39
3.5	Architectural Design	39
3.5.1	Physical Architecture	39
3.5.2	Data Flow Architecture	39
3.6	Technology Choices	40
3.6.1	Why ONNX Runtime	40
3.6.2	Why Physics-Based Simulation over Pure QEMU Emulation	40
3.6.3	Why the Two-Pass Calibration Strategy	40
3.6.4	Simulation Parameter Choices	40
3.7	Comparative Results	41
3.7.1	Methodology Comparison	41
3.7.2	Key Findings	41
3.7.3	Reproducibility	42
4	Réalisation	43
4.1	Choix techniques	44
4.2	Travail réalisé	44
4.3	Planning réel du projet	44
	Conclusion générale	45
	Bibliography	46
	Annexes	47
	Annexe 1. Exemple d'annexe	47
	Annexe 2. Entreprise	48

List of Figures

1.1	Logo of the Barcelona Supercomputing Center.	3
1.2	Logo of the High Performance and Embedded Systems Laboratory (HPES).	4
3.1	Shared four-stage pipeline common to all three methodologies.	19
3.2	Tiny-ImageNet preprocessing pipeline applied identically on every inference path. . .	20
3.3	Producer-consumer inference pipeline for Methodology 1.	22
3.4	Latency distribution — ResNet50-v1.5.	25
3.5	Latency distribution — MobileNetV2.	26
3.6	Latency distribution — MobileNetV3-Small.	26
3.7	Latency distribution — MobileNetV1.	27
3.8	Latency distribution — EfficientNet-Lite0.	27
3.9	Latency distribution — SSD-MBv2-backbone.	28
3.10	Preprocessing vs. inference time breakdown (x86 CPU baseline).	28
3.11	Latency distribution — ResNet50-v1.5.	35
3.12	Latency distribution — MobileNetV2.	35
3.13	Latency distribution — MobileNetV3-Small.	35
3.14	Latency distribution — MobileNetV1.	35
3.15	Latency distribution — EfficientNet-Lite0.	36
3.16	Latency distribution — SSD-MBv2-backbone.	36
Annexe 2.1	Logo d’entreprise	48

List of Tables

3.1	Model registry and characterisation.	19
3.2	Host hardware and software environment for Methodology 1.	24
3.3	Latency and throughput results — x86 CPU baseline.	24
3.4	NXP i.MX 8M Plus hardware specification targeted by the simulation.	29
3.5	Per-model simulation parameters.	33
3.6	Tri-path mean latency comparison (Methodology 2).	33
3.7	Summary of Methodology 3 outcomes.	39
3.8	Summary comparison of the three benchmarking methodologies.	41
Annexe 1.1 Exemple tableau dans l'annexe		47

Liste des abréviations

- **GISI** = Génie Informatique des Systèmes Industriels
- **GLSI** = Génie Logiciel et Systèmes d'Information
- **GTR** = Génie des Télécommunications et Réseaux

General introduction

Exemple d'utilisation de la bibliographie utilisée [?]. Le style utilisé est IEEE [?].

Une introduction dure à 3 pages où vous poserez clairement le problème auquel vous allez tenter d'apporter une solution. L'introduction se rédige à la fin de votre travail de rédaction. Avant de rédiger l'introduction, structurez TOUT le PFE. L'introduction peut se faire en même temps que la conclusion.

L'introduction sert trois objectifs :

- elle introduit le sujet. Ceci signifie qu'il faut présenter succinctement le contexte général du travail accompli, par exemple l'environnement professionnel et l'entreprise pour un rapport de stage, puis définir le sujet en termes précis et concis;
- elle énonce ensuite succinctement les objectifs du travail personnel, et les moyens mis en œuvre pour tenter de les atteindre;
- elle s'achève sur une présentation claire du plan adopté pour la suite du corps du rapport. L'annonce du plan se fait au futur et doit être rédigée en entier.

L'introduction générale doit développer les points suivants :

- la présentation du contexte du projet (domaine exemple : télécommunication, sécurité, automate etc.);
- la présentation brève de l'entreprise d'accueil et de son domaine;
- la description des objectifs du PFE/ Mémoire : justifier le sujet et poser le problème à résoudre; indiquer la manière dont il sera traité en termes de outils et de méthodes; donner les raisons qui président à ce choix; exposer les intérêts du sujet et sa problématique;
- l'annonce du plan du rapport sans trop détailler. Il est recommandé, à partir de l'introduction générale, de recourir au *résumé* de modeste.

General context of the project

Contents

1	Host organisation	3
2	Problem statement	4
3	Solution approach	6
4	Presentation of the proposed solution	6
5	Work process and documentation	7

Introduction

This introductory chapter presents the host organization and its diverse areas of activity. It then formulates the problem statement and outlines the proposed solution. Finally, the chapter describes the project methodology and provides insights into the adopted approach.

1.1 Host organisation

Within this section, we are going to introduce the host organisation and hierarchy of it, as well, introduce the research lab where this internship took place.

1.1.1 Barcelona super computing center

The Barcelona Supercomputing Center (BSC-CNS) is Spain's national supercomputing facility, located in Barcelona. It hosts MareNostrum, one of the most powerful supercomputers in Europe, and serves as a leading research centre in high performance computing (HPC), computer sciences, and computational engineering. BSC-CNS is a member of the Partnership for Advanced Computing in Europe (PRACE) and actively contributes to European research initiatives. Its mission includes providing HPC resources, developing advanced software and tools, and fostering technology transfer to both industry and academia. Figure 1.1 shows the official logo of BSC-CNS.

The centre is organised into several departments, among which the Computer Sciences (CS) department and the Earth Sciences department are particularly prominent. The work presented in this report was carried out within the **High Performance and Embedded Systems Laboratory (HPES)**, which belongs to the Computer Sciences department of BSC-CNS.



Figure 1.1: Logo of the Barcelona Supercomputing Center.

1.1.2 High Performance and Embedded Systems Laboratory (HPES)

The High Performance and Embedded Systems Laboratory (HPES) at BSC-CNS focuses on the design, analysis, and optimisation of computing systems ranging from large scale parallel machines to embedded real time devices. Key research areas include:

- Parallel programming models and runtime systems.

- Performance analysis and tuning for HPC applications.
- Energy efficient and embedded architectures.
- Fault tolerance and reliability in extreme scale systems.

HPES brings together experts in computer architecture, compilers, operating systems, and application development. The laboratory collaborates on European projects (e.g., EPI, EuroHPC) and with industrial partners to transfer its innovations into practice. Figure 1.2 presents the HPES lab logo.



Figure 1.2: Logo of the High Performance and Embedded Systems Laboratory (HPES).

The present work was conducted within the **SODES** group of HPES, which specialises in the design and optimisation of embedded systems for edge computing and autonomous devices.

1.2 Problem statement

The rapid proliferation of intelligent edge devices from autonomous robots and drones to smart cameras and industrial sensors has created an unprecedented demand for on-device artificial intelligence (Edge AI). Unlike cloud-based AI, Edge AI requires low latency, bandwidth efficiency, data privacy, and energy autonomy. These constraints push computational tasks away from centralised data centres towards resource constrained embedded platforms.

1.2.1 The computational bottleneck of Edge AI

Modern AI models, particularly deep neural networks (DNNs), are computationally intensive. Running them on general purpose CPUs leads to excessive inference times and high energy consumption, often violating real time requirements. While GPUs offer high throughput, their power profile (typically tens to hundreds of watts) makes them unsuitable for battery powered or passively cooled edge devices.

1.2.2 Hardware acceleration as the key enabler

To address this gap, system on chip (SoC) vendors have begun integrating dedicated Neural Processing Units (NPUs) which are specialised accelerators designed to execute tensor operations, convolutions, and activation functions with extreme efficiency. An NPU can deliver 10 to 100 times better performance per watt than a CPU for typical DNN workloads. Examples include the Ethos U series from Arm, the NPU inside the Rockchip RK3588, and the Neural Engine in Apples A series chips.

1.2.3 The configuration challenge

However, leveraging an NPU is not a matter of simple integration. Each NPU has unique constraints: tiling of tensors into a small local memory (scratchpad), data layout (NHWC vs. NCHW), quantisation schemes (int8, int16, or binary), and supported operations (e.g., depthwise convolutions, activations). Furthermore, the overall system performance depends on the interplay between the NPU, the CPU (for control and preprocessing), the memory hierarchy, and the communication bus.

The core problem is therefore:

How can we determine the optimal configuration (operator partitioning, memory tiling, quantisation, and task scheduling) of a dedicated NPU based heterogeneous system to maximise the acceleration of Edge AI workloads compared to a CPU only baseline, while meeting real time and energy constraints?

1.2.4 Relevance to this work

Within the SODES group at HPES (BSC-CNS), this problem is particularly relevant. Our target embedded platform integrates a CPU cluster and a dedicated NPU. Without proper configuration, the NPU may be underutilised or even slower than a well optimised CPU implementation due to data transfer overhead or mismatched tensor layouts.

This work aims to:

- Characterise the performance gap between CPU only and NPU accelerated execution for representative Edge AI models (object detection, classification models, etc . . .).
- Develop a systematic methodology to explore the configuration space and identify Pareto optimal configurations based on the different approaches and standards existing in this field such as The Benchmarking Standard: The MLPerf Tiny per exemple.

- Implement and validate the configuration on real hardware, measuring end to end latency, energy, and throughput.

By solving this configuration problem, we enable practical Edge AI deployment that fully exploits the NPUs potential, turning the promise of hardware acceleration into measurable system level gains.

1.3 Solution approach

The project follows a two stage implementation strategy. First, an initial implementation is carried out on top of the QEMU emulator. This allows us to evaluate the performance of the system across different simulated platforms and to develop a physical model that estimates the latency and throughput of CPU based inference. The emulation on an x86 host provides a controlled environment for early validation.

Second, the emulation is ported to the real hardware platform. This step yields actual performance measurements under realistic operating conditions. Based on these measurements, various optimisation techniques for NPU inference latency are applied. The goal is to identify the configuration that achieves the optimal performance of which the platform is capable.

1.4 Presentation of the proposed solution

The proposed work is built upon successive implementations of the target system. Initially, we simulate the platform using the QEMU emulator. This first implementation serves two purposes: it provides a baseline for CPU based inference through physical emulation, and it enables the development of a performance model that computes the system state and estimates latency and throughput.

Subsequently, we port the system to the actual embedded platform. Inference is executed on the real hardware, allowing a comparative analysis between the general purpose computing units (CPU) and the dedicated machine learning accelerator (NPU). The comparison focuses on inference latency, energy consumption, and throughput, thereby quantifying the benefits of NPU acceleration.

1.5 Work process and documentation

The project follows a structured workflow comprising requirement analysis, iterative development, experimental validation, and documentation. Each phase is documented in the corresponding chapters of this report. Version control is maintained using Git, and experimental results are logged systematically to ensure reproducibility. A detailed project schedule is provided in Appendix A (if applicable).

Conclusion

This chapter has presented the host organisation and the HPES laboratory, formulated the problem of NPU configuration for Edge AI, and described the proposed a two-staged solution approach. The next chapter will delve into the requirements analysis and specification for the project.

Requirements Analysis and Operating Environment

Contents

1	Requirements Analysis	9
2	System Architecture	11
3	System Modeling	13
4	Work Environment	13

Introduction

In this chapter, we define the functional and non-functional requirements of our Edge AI benchmarking system, which aims to measure the performance gains of running ONNX model inference on the Vivante GC7000L NPU compared to the CPU on the NXP i.MX 8M Plus platform. We then present the system architecture, describing the interaction between the host PC, the target board, and the benchmarking pipeline components. System modeling is introduced with placeholders for diagrams to be finalized. Finally, we detail the hardware and software environment used throughout this project, from emulation to physical deployment.

2.1 Requirements Analysis

Requirements analysis is a fundamental phase in any research and development project. It defines what the system must do and how it should perform under given constraints. In our context, the system is not a user-facing application but an automated benchmarking pipeline designed to evaluate and compare inference performance across different processing units on heterogeneous SoCs. This section covers both functional and non-functional requirements.

2.1.1 Functional Requirements

A functional requirement defines the specific behaviors, functions, and capabilities the system must exhibit. For our benchmarking system, the functional requirements are organized into the following categories:

Benchmark Execution

- **Inference on CPU:** The system must run ONNX model inference using the CPU execution provider of ONNX Runtime on the i.MX 8M Plus.
- **Inference on NPU:** The system must run ONNX model inference using the VSI NPU execution provider, targeting the Vivante GC7000L NPU on the i.MX 8M Plus.
- **Sequential execution:** The system must execute CPU and NPU benchmarks separately and sequentially to avoid resource contention and ensure clean measurements.
- **Model support:** The system must support loading and executing deep learning models in ONNX format, including model variants such as MobileNet and YOLO.

Metrics Collection

- **Latency measurement:** The system must measure per-inference latency in milliseconds for each execution provider.
- **Throughput measurement:** The system must measure throughput in inferences per second over a defined batch of runs.
- **Thermal monitoring:** The system must collect temperature data from the SoC during benchmark execution to assess thermal behavior.
- **PMU register access:** The system must read Performance Monitoring Unit (PMU) registers to extract low-level hardware performance counters (e.g., cycles, instructions, cache events) for deeper architectural insights.

Result Handling

- **JSON logging:** The system must store all collected metrics (latency, throughput, temperature, PMU counters) in structured JSON files for later processing.
- **SCP transfer:** The system must allow transfer of result JSON files from the target board to the host PC via SCP for visualization and analysis.

Visualization and Analysis

- **Comparative visualization:** The host PC must provide scripts or tools to parse JSON results and generate comparative graphs (e.g., bar charts, line plots) showing CPU vs NPU performance.
- **Comparative analysis:** The system must enable side-by-side analysis of latency, throughput, thermal, and PMU metrics across execution providers.

Automation

- **Shell-based pipeline:** The entire benchmarking workflow from model loading to result storage must be automated via shell scripts for sequential, reproducible execution.

2.1.2 Non-Functional Requirements

Non-functional requirements specify quality attributes and constraints that the system must satisfy. They are critical for ensuring the validity, reproducibility, and rigor of our benchmarking results.

- **Accuracy trade-off analysis:** The system should allow comparison of inference accuracy across CPU and NPU execution providers to identify any accuracy degradation due to hardware-specific optimizations or quantization. Quantifying the accuracy-latency trade-off is a core objective.
- **Reproducibility:** All benchmark runs must be reproducible. This includes documenting compiler flags, runtime configurations, execution provider settings, model quantization parameters, and environmental conditions. The methodology should align with established benchmarking standards.
- **Adherence to standards:** The benchmarking methodology should follow the principles and best practices defined by:
 - **MLPerf Tiny** [?] for standardized TinyML benchmarking workloads and measurement rules.
 - **EEMBC** [?] for edge-focused benchmark guidelines (to be explored during the project).
- **Overhead measurement:** The system must measure and account for any instrumentation overhead, including the impact of PMU register sampling and thermal polling on inference timing.
- **Portability:** The benchmarking framework should be designed with portability in mind. While the primary target is the NXP i.MX 8M Plus, the pipeline should be adaptable to other heterogeneous SoCs if time permits.
- **Hardware constraint INT8 Quantization:** The Vivante GC7000L NPU on the i.MX 8M Plus supports only INT8 quantized models. Consequently, all models deployed on the NPU execution provider must be quantized to INT8 precision. This hardware limitation is a central constraint of our system.

2.2 System Architecture

In this section, we present an abstraction of the system architecture, detailing the main modules and their interactions. Our system is composed of a host PC for development, automation, and visualization, and the NXP i.MX 8M Plus target board where benchmarks are executed. Communication between the two is handled via SCP. Figure ?? presents an overview of this architecture.

2.2.1 Host PC

The host PC runs Ubuntu Linux and serves three main purposes:

- **Development environment:** Cross-compilation of ONNX Runtime with the VSI NPU backend, Yocto Linux image generation, and benchmarking script development.
- **Automation control:** The PC initiates the benchmarking pipeline by triggering shell scripts on the target board (via SSH or serial connection) and retrieves results.
- **Visualization:** Python-based scripts parse the JSON result files and generate comparative graphs and reports.

2.2.2 Target Board NXP i.MX 8M Plus

The i.MX 8M Plus is the core execution platform. It hosts the following components:

- **ONNX Runtime:** The inference engine, compiled with two execution providers:
 - **CPU Execution Provider:** Leverages the ARM Cortex-A cores for floating-point inference.
 - **VSI NPU Execution Provider:** Offloads INT8-quantized model inference to the Vivante GC7000L NPU.
- **Benchmark Harness:** A set of shell scripts that automate the sequential execution of benchmarks, including warm-up runs, timed inference loops, and result collection.
- **Metrics Collection Module:** A combination of tools and scripts responsible for:
 - Reading PMU registers (via `perf` or custom kernel interfaces).
 - Polling thermal sensors.
 - Measuring inference latency and throughput.
- **JSON Logger:** Serializes all collected metrics into structured JSON files stored locally on the board.
- **Yocto Linux:** A custom-built lightweight Linux distribution tailored for the i.MX 8M Plus, providing the necessary kernel drivers for NPU and PMU access.

2.2.3 QEMU Emulation Layer

During the initial phase of the project, a QEMU-based emulation environment was used to simulate the i.MX 8M Plus platform. This allowed early development and validation of the benchmarking pipeline before physical hardware was available. Mathematical models and scaling factors were developed to project emulated results toward expected physical hardware performance. The emulation methodology is detailed in Chapter 3.

2.2.4 Data Flow

The benchmarking data flow proceeds as follows:

- ONNX models are deployed to the i.MX 8M Plus filesystem.
- The benchmark harness launches inference with a specified execution provider (CPU or NPU).
- During inference, the metrics collection module samples PMU registers and thermal data.
- Upon completion, latency, throughput, temperature, and PMU counter values are written to a JSON file.
- JSON files are transferred from the target board to the host PC via SCP.
- On the host PC, Python scripts parse the JSON files and generate comparative visualizations.

2.3 System Modeling

2.4 Work Environment

This section details the hardware and software components constituting the work environment for this project. The environment spans two main phases: an initial emulation phase using QEMU and a subsequent deployment phase on the physical NXP i.MX 8M Plus hardware.

2.4.1 Hardware Environment

The hardware environment is composed of a host development PC and the target embedded platform.

2.4.1.1 Host PC

The host PC is a standard x86_64 machine running Ubuntu Linux. It is used for cross-compilation, Yocto build orchestration, and result visualization. (Detailed specifications to be confirmed.)

- **OS:** Ubuntu 22.04.5 LTS x86_64 (Kernel 6.8.0-124-generic)
- **CPU:** Intel Ultra 7 165U (14 cores) @ up to 4.900 GHz
- **RAM:** 16 GiB .
- **Storage:** 512 GO .

2.4.1.2 Target Board NXP i.MX 8M Plus

The NXP i.MX 8M Plus [?] is a heterogeneous SoC designed for Edge AI applications. Its key components relevant to our project are:

- **CPU:** Quad-core ARM Cortex-A53
- **NPU:** Vivante GC7000L, supporting INT8 quantized inference
- **GPU:** Vivante GC7000UL (not the primary focus of this study)
- **RAM:** LPDDR4 (*TBD* GB)
- **Storage:** eMMC / SD card (*TBD*)
- **Interfaces:** Gigabit Ethernet, USB, GPIO, UART

2.4.1.3 Emulation Platform QEMU

During the software emulation phase, QEMU was configured to emulate the ARM Cortex-A53 cores of the i.MX 8M Plus. This allowed early benchmarking pipeline development and performance projection via mathematical modeling.

2.4.2 Software Environment

The software stack is built around open-source tools and frameworks for embedded Linux, inference, and data analysis.

- **Yocto Linux:** A custom Linux distribution built using the Yocto Project for the i.MX 8M Plus. It includes necessary kernel modules for NPU (Vivante driver stack) and PMU access.

- **QEMU:** Used for full-system emulation of the target ARM64 architecture during the initial project phase.
- **ONNX Runtime:** The primary inference engine, built from source for ARM64 with the VSI NPU execution provider enabled for NPU offloading.
- **VSI NPU Backend:** The Vivante NPU driver and runtime stack enabling ONNX Runtime to offload INT8 inference to the GC7000L NPU.
- **Python:** Used for host-side automation scripting, JSON parsing, and visualization.
- **Shell/Bash:** Core scripting language for the benchmark harness and automation pipeline on the target board.
- **Visualization Libraries:** (*TBD* Matplotlib, Plotly, or similar) for generating comparative graphs.
- **Cross-compilation Toolchain:** ARM64 GCC toolchain (aarch64-none-linux-gnu) for building ONNX Runtime and other native components.
- **PMU Tools:** Linux `perf` subsystem and/or custom kernel interfaces for accessing hardware performance counters.

Conclusion

In this chapter, we defined the functional requirements of our Edge AI benchmarking system, focusing on the automated execution of ONNX model inference on CPU and NPU, comprehensive metrics collection (latency, throughput, thermal, PMU counters), JSON-based result logging, and comparative visualization. We also specified the non-functional requirements, emphasizing reproducibility, adherence to MLPerf Tiny and EEMBC standards, accuracy trade-off analysis, and the INT8 quantization constraint imposed by the Vivante GC7000L NPU.

The system architecture was presented, detailing the host PC, the i.MX 8M Plus target board with its dual execution providers (CPU and VSI NPU), the QEMU emulation layer, and the data flow from benchmark execution to visualization. System modeling diagrams were reserved as placeholders pending discussion with the project supervisor.

Finally, we detailed the hardware and software work environment, covering the host PC, the i.MX 8M Plus board, QEMU, and the full software stack from Yocto Linux to ONNX Runtime and visualization tools.

In the next chapter, we will present the software emulation phase, detailing the QEMU setup, the mathematical models used for performance projection, and the initial benchmarking results obtained through emulation before transitioning to physical hardware.

Emulation and Simulation Framework

Contents

1	Shared Pipeline Infrastructure	18
2	Methodology 1: x86 CPU Baseline	21
3	Methodology 2: Physics-Based ARM and NPU Simulation	29
4	Methodology 3: QEMU User-Mode ARM64 Emulation	37
5	Architectural Design	39
6	Technology Choices	40
7	Comparative Results	41

Introduction

Before deploying inference workloads on the physical NXP i.MX 8M Plus board, it is essential to establish a reproducible performance baseline and to develop a realistic estimation of what the target hardware can deliver. This chapter presents a three-methodology framework that progressively builds from a measured x86 CPU baseline, through a physics-based simulation of the ARM Cortex-A53 CPU and Vivante GC7000UltraLite NPU, to an attempted QEMU user-mode ARM64 emulation. Together, these methodologies provide a comprehensive pre-hardware characterisation of the six ONNX models introduced in Chapter 2, enabling informed decisions about model selection and deployment strategy before physical access to the target platform is available.

The chapter is organised as follows. Section 3.1 presents the shared pipeline infrastructure common to all three methodologies. Section 3.2 details Methodology 1, the x86 CPU baseline. Section 3.3 describes Methodology 2, the physics-based tri-path simulation. Section 3.4 documents Methodology 3, the QEMU emulation attempt and the reasons for its abandonment. Section 3.5 presents the architectural design of the emulation environment. Section 3.6 justifies the key technology choices and simulation parameters. Finally, Section 3.7 summarises the comparative results across methodologies.

3.1 Shared Pipeline Infrastructure

All three methodologies share a common four-stage pipeline skeleton that ensures consistency in model handling, data flow, and metric computation.

3.1.1 Four-Stage Pipeline Structure

Every benchmarking run, regardless of methodology, follows the same four-stage structure:

- **Setup:** Model export to ONNX opset 17 with weight integrity verification, and dataset construction from Tiny-ImageNet-200.
- **Inference:** A pipelined producer-consumer architecture where preprocessing and inference are decoupled, with isolated inference timing.
- **Metrics:** Statistical computation of latency distributions (mean, median, p95, p99, standard deviation, IQR, coefficient of variation), throughput, and top-1 accuracy for classification models.

- **Visualisation:** Generation of latency-over-time traces, histogram/KDE distributions, comparative bar charts, and accuracy–latency Pareto scatter plots.

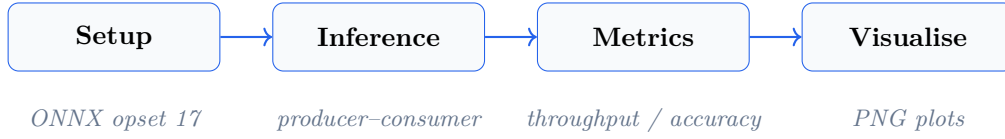


Figure 3.1: Shared four-stage pipeline common to all three methodologies.

3.1.2 Model Zoo

Six ONNX models are evaluated across all methodologies. Table 3.1 summarises their characteristics.

Table 3.1: Model registry and characterisation.

Model	Input	Classes	Task	Source	Min MB	Top-1
ResNet50-v1.5	224 ²	1000	cls	torchvision	90	≈ 76%
MobileNetV2	224 ²	1000	cls	torchvision	12	≈ 72%
MobileNetV3-Small	224 ²	1000	cls	torchvision	8	≈ 67%
MobileNetV1-1.0	224 ²	1000	cls	timm	14	≈ 71%
EfficientNet-Lite0	224 ²	1000	cls	timm	15	≈ 74%
SSD-MBv2-backbone	300 ²	—	det	torchvision	10	N/A

timm dependency. Only MobileNetV1 and EfficientNet-Lite0 require the `timm` library (`pip install timm`). All other models depend only on `torch`, `torchvision`, `onnxruntime`, `numpy`, and `pillow`.

3.1.3 Dataset Construction

3.1.3.1 Tiny-ImageNet-200 Validation Set

Tiny-ImageNet-200 is a 200-class subset of ImageNet distributed by Stanford CS231n [?]. The validation partition contains 10 000 JPEG images (6464 pixels, 50 per class), annotated with WordNet IDs (WNIDs). A handcrafted lookup table ϕ maps each WNID to its canonical ImageNet-1k class index (0–999):

$$\phi : \text{WNID} \rightarrow \{0, \dots, 999\} \subset \mathbb{Z} \quad (3.1)$$

All 10 000 validation images are selected with a fixed random seed (`seed=42`). The archive (≈ 236 MB) is cached locally so that subsequent runs skip the download.

3.1.3.2 Preprocessing Pipeline

The models were pre-trained on full ImageNet at 224224 pixels. Since Tiny-ImageNet images are natively 6464, a three-step preprocessing pipeline is applied identically across all inference paths:



Figure 3.2: Tiny-ImageNet preprocessing pipeline applied identically on every inference path.

- **Bilinear resize:** The image is resized so that its shorter dimension equals 256 px, preserving aspect ratio. PIL BILINEAR interpolation is used, consistent with the standard ImageNet evaluation protocol.
- **Centre crop to 224224:** A 224224 region is extracted from the centre of the 256256 intermediate. The crop offsets are:

$$\text{left} = \lfloor (W - 224)/2 \rfloor, \quad \text{top} = \lfloor (H - 224)/2 \rfloor \quad (3.2)$$

where W and H are the width and height of the resized image.

- **Normalisation:** Each pixel value is scaled to $[0, 1]$ by dividing by 255, then standardised channel-wise:

$$x'_c = \frac{x_c/255 - \mu_c}{\sigma_c}, \quad c \in \{R, G, B\}, \quad \mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225] \quad (3.3)$$

The output is a `float32` NumPy array of shape $[1, 3, 224, 224]$, memory footprint ≈ 0.60 MB. This tensor’s DMA transfer from LPDDR4 into NPU SRAM constitutes the fixed overhead $\Delta_{\text{DMA}} = 0.18$ ms in the NPU simulation (Section 3.3.5).

Impact on accuracy. Upsampling from 64*64 to 224*224 introduces bilinear blur not present during training. Top-1 accuracy is consequently 16%–31% versus the 69%–80% these models achieve on genuine ImageNet data. Accuracy is treated as a *sanity check*: values well above the

$\approx 0.1\%$ random baseline confirm correct model loading, label remapping, and preprocessing integrity. Latency measurements the primary benchmark output are unaffected by the resolution mismatch.

3.2 Methodology 1: x86 CPU Baseline

3.2.1 Overview

Methodology 1 establishes a reproducible performance baseline by executing unmodified `float32` ONNX models on an AMD Ryzen 5 5600H using ONNX Runtime’s `CPUExecutionProvider`. This serves two purposes: (1) confirming that pretrained weights are correctly loaded and characterising accuracy under the Tiny-ImageNet resolution mismatch, and (2) providing reference latency measurements for relative comparison with the simulation-based methodologies.

3.2.2 ONNX Runtime Session Configuration

Sessions are created with full graph optimisation enabled. The number of intra-op and inter-op threads is set to 0 (auto), allowing ONNX Runtime to utilise all available physical cores for maximum parallelism.

```
1 opts = ort.SessionOptions()
2 opts.intra_op_num_threads      = 0    # auto = all physical cores
3 opts.inter_op_num_threads      = 0
4 opts.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL
5 opts.enable_profiling          = False
6
7 session = ort.InferenceSession(
8     model_path,
9     sess_options=opts,
10    providers=["CPUExecutionProvider"]
11 )
```

Listing 3.1: ONNX Runtime session creation with full graph optimisation.

3.2.3 Producer–Consumer Pipeline

To prevent preprocessing from inflating inference timing, a producer–consumer architecture decouples JPEG decode/resize from ONNX session execution:

- **Producer:** A `ThreadPoolExecutor` with 4 workers preprocesses images and submits ready

tensors to a bounded queue (capacity 32). The producer maintains the original image index, preserving the link between image, tensor, and ground-truth label.

- **Consumer:** The main thread pulls pre-built tensors from the queue and calls `session.run()` sequentially, recording per-inference latency.

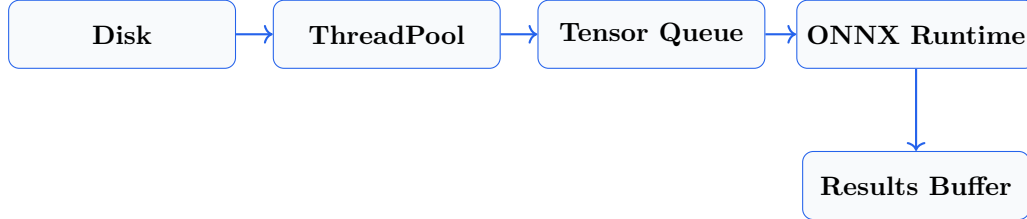


Figure 3.3: Producer-consumer inference pipeline for Methodology 1.

3.2.4 Timing Isolation

Only the `session.run()` call is timed using `time.perf_counter()`:

$$t_{\text{inf},i} = t_1 - t_0, \quad t_0 = \text{perf_counter() before}, \quad t_1 = \text{perf_counter() after} \quad (3.4)$$

Preprocessing time is recorded separately as an overlapped cost and does not contribute to the inference latency histogram.

3.2.5 Warm-up Protocol

Five initial inference calls are executed before timing begins and excluded from all latency statistics. This protocol: (1) completes ONNX Runtime JIT compilation, (2) warms CPU instruction caches and the TLB, and (3) initialises BLAS thread pools and numerical libraries.

3.2.6 Inference Algorithm

Algorithm 1 Methodology 1 Baseline Inference Benchmark

Require: Model $\mathcal{M}$, dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=0}^{N-1}$, $W = 5$, $Q = 16$, $P = 4$ **Ensure:** Latency vector $\mathbf{L} \in \mathbb{R}^N$, accuracy A

```

1: Create ORT session  $S$  with ORT_ENABLE_ALL
2: for  $w = 1 \dots W$  do
3:    $S.run(preprocess(x_0))$  ▷ warm-up, discarded
4: end for
5: Start producer with  $P$  workers, queue capacity  $2Q$ 
6: for  $i = 0 \dots N - 1$  do
7:    $(i', \hat{x}, t_{pp}) \leftarrow Q.get()$ 
8:    $t_0 \leftarrow perf\_counter()$ 
9:    $\hat{y} \leftarrow S.run(\hat{x})$ 
10:   $t_1 \leftarrow perf\_counter()$ 
11:   $L_{i'} \leftarrow (t_1 - t_0)1000$  ▷ ms
12:  if task = cls then
13:    correct $_{i'} \leftarrow \mathbf{1}[\arg \max \hat{y} = y_{i'}]$ 
14:  end if
15: end for
16:  $A \leftarrow \frac{1}{N} \sum correct_i 100$  if cls else NONE
17: return  $\mathbf{L}$ ,  $A$ 

```

3.2.7 Statistical Metrics

For each model, the latency vector $\mathbf{L} = (L_0, \dots, L_{N-1})$ in milliseconds yields:

- **Mean:** $\bar{L} = \frac{1}{N} \sum L_i$
- **Median (p50):** $L_{0.50}$, the typical latency
- **p95:** $L_{0.95}$, the SLA tail
- **p99:** $L_{0.99}$, the strict tail used in MLPerf
- **Std:** $\sigma = \sqrt{\frac{1}{N} \sum (L_i - \bar{L})^2}$

- **IQR:** $L_{0.75} - L_{0.25}$
- **Throughput:** $T = 1000/\bar{L}$ img/s
- **CV:** $\sigma/\bar{L}$; < 0.10 stable, $\in [0.10, 0.30)$ moderate jitter, ≥ 0.30 high jitter

3.2.8 Experimental Setup

Table 3.2: Host hardware and software environment for Methodology 1.

Parameter	Value
Hardware	x86-64 AMD Ryzen 5 5600H
Runtime	ONNX Runtime <code>CPUExecutionProvider</code>
Batch size	1 (sequential)
Warm-up runs	5 (excluded from timing)
Preprocessing workers	4 threads (<code>ThreadPoolExecutor</code>)
Queue depth	32 tensors
Dataset	Tiny-ImageNet-200 validation, $N = 10,000$
Label space	ImageNet-1k (0–999)

3.2.9 Results

3.2.9.1 Latency and Throughput

Table 3.3: Latency and throughput results — x86 CPU baseline.

Model	Task	$\bar{L}$ (ms)	p50	p95	p99	σ	CV	img/s
ResNet50-v1.5	cls	22.74	19.24	41.37	66.39	9.52	0.419	43.98
MobileNetV2	cls	4.58	4.15	7.68	11.82	1.96	0.429	218.52
MobileNetV3-Small	cls	3.45	3.43	5.32	7.35	1.20	0.348	289.89
MobileNetV1-1.0	cls	6.05	5.20	11.06	17.89	2.80	0.464	165.39
EfficientNet-Lite0	cls	5.45	4.85	9.31	14.37	2.39	0.438	183.43
SSD-MBv2-backbone	det	7.87	6.81	13.51	24.72	3.69	0.469	127.01

3.2.9.2 Visualisation Suite

Latency over time. Per-image latency trace overlaid with mean $\bar{L}$, p95, and $\pm 1\sigma$ band. A flat trace indicates thermal and frequency stability; a rising trend indicates thermal throttling; spikes indicate OS scheduling jitter.

3.2.10 Latency Distribution: Histogram and KDE Visualisation

Combined histogram and Gaussian KDE with Scott's bandwidth rule $h = n^{-1/5} \cdot \sigma$. Mean and p95 are marked with dashed/dotted vertical lines. A narrow tall peak indicates consistent performance; a heavy right tail signals occasional stalls.

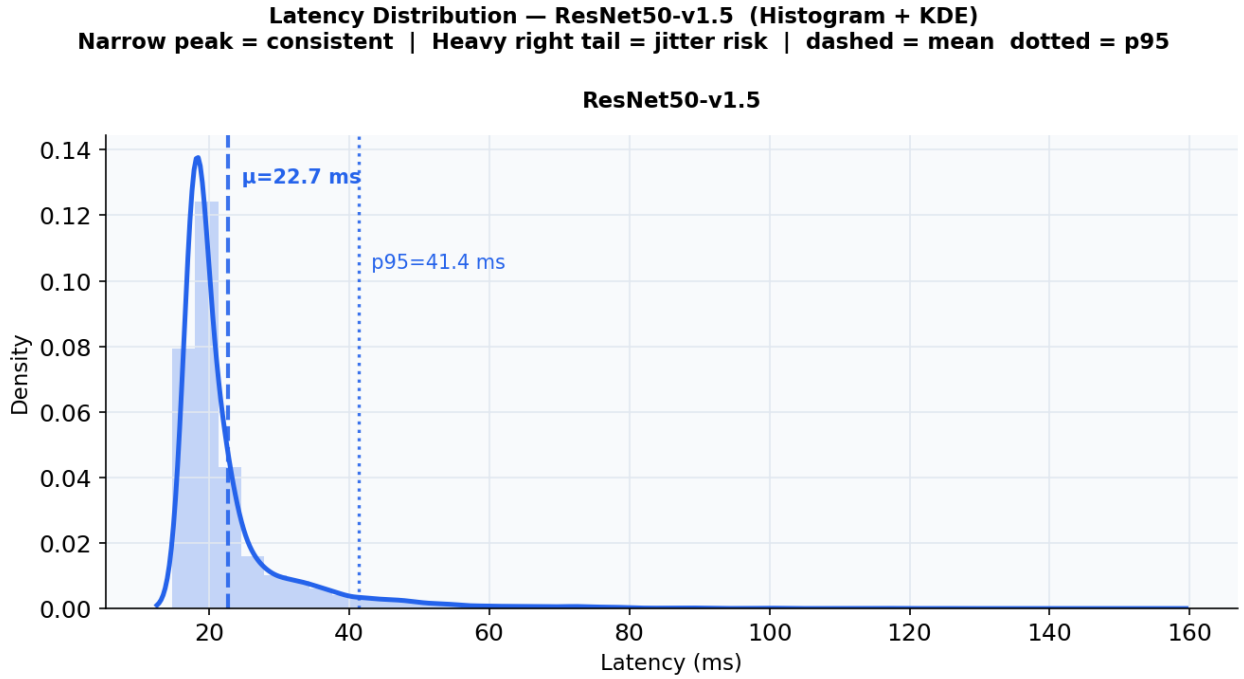


Figure 3.4: Latency distribution — ResNet50-v1.5.

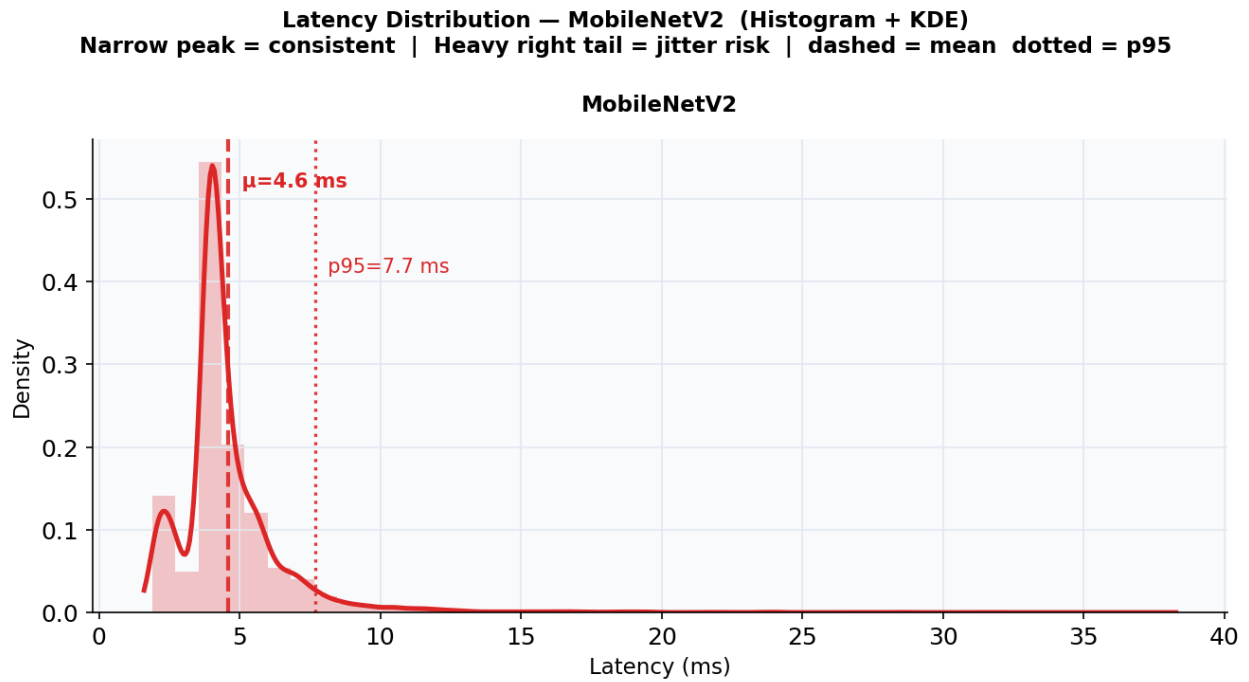


Figure 3.5: Latency distribution — MobileNetV2.

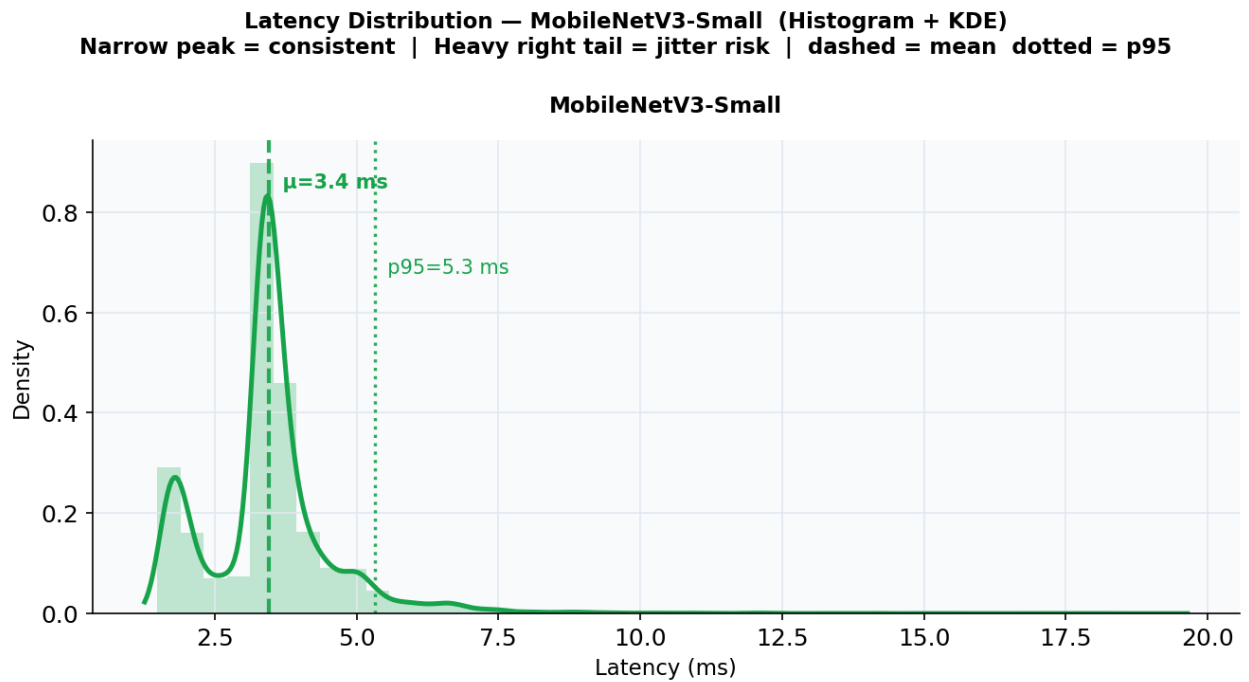


Figure 3.6: Latency distribution — MobileNetV3-Small.

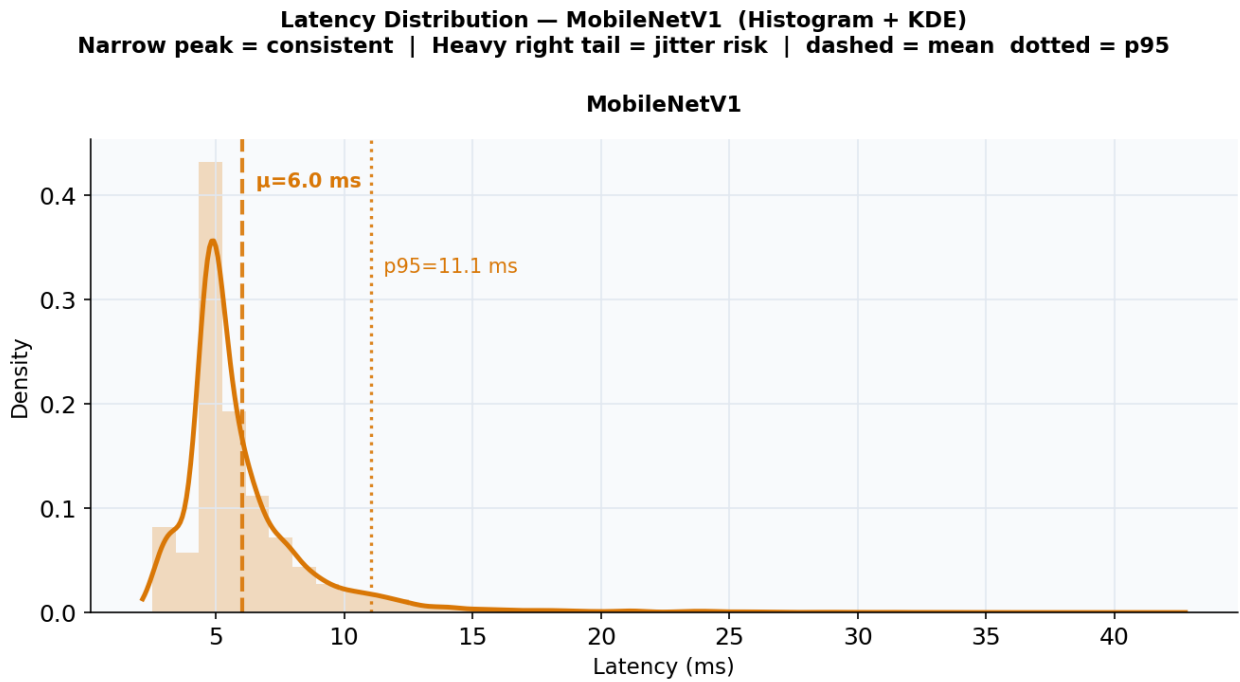


Figure 3.7: Latency distribution — MobileNetV1.

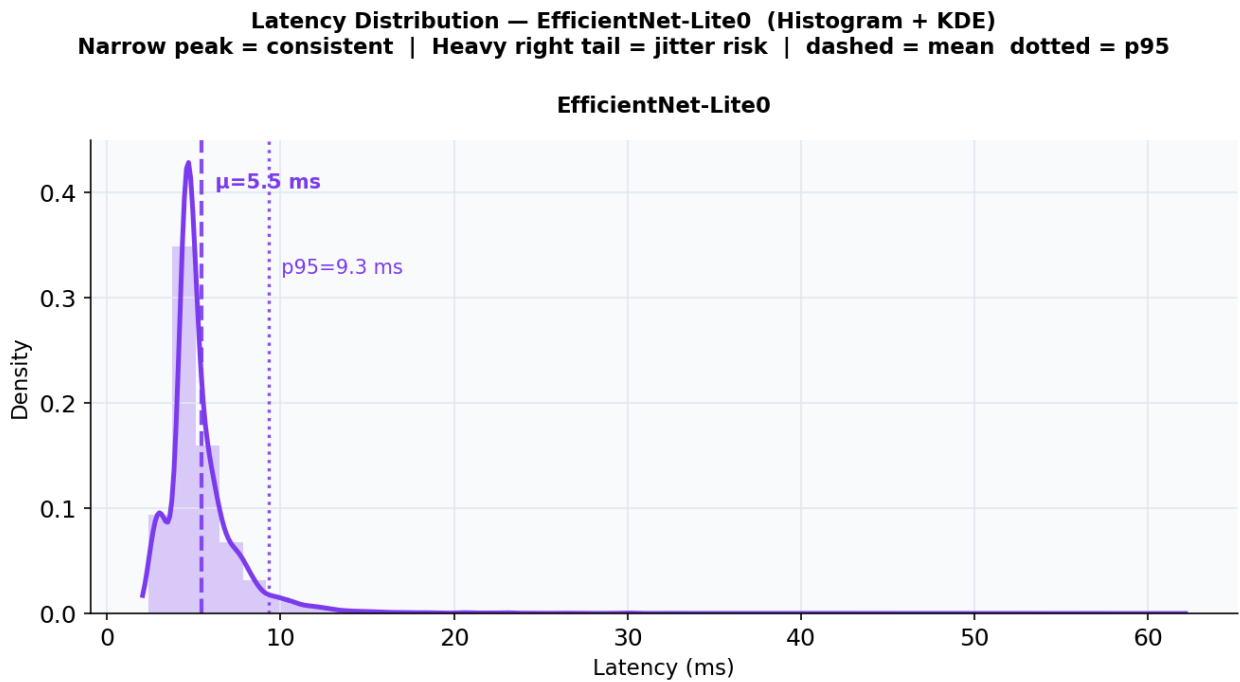


Figure 3.8: Latency distribution — EfficientNet-Lite0.

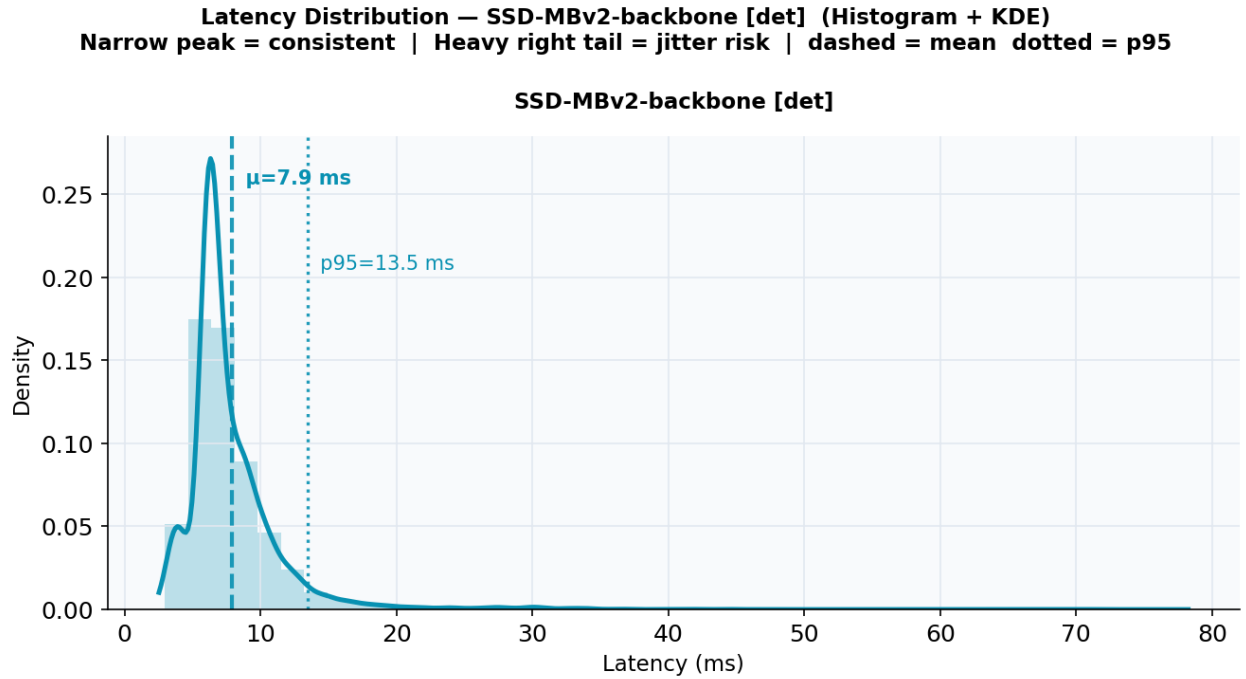


Figure 3.9: Latency distribution — SSD-MBv2-backbone.

Arm vs x86 projection . within this picture below, we can have an estimation, based on the CPU power that we have run our system, and the computing power of the main cpu of the imx8m plus, we can project those values as a prenminal estimation for the system preformance

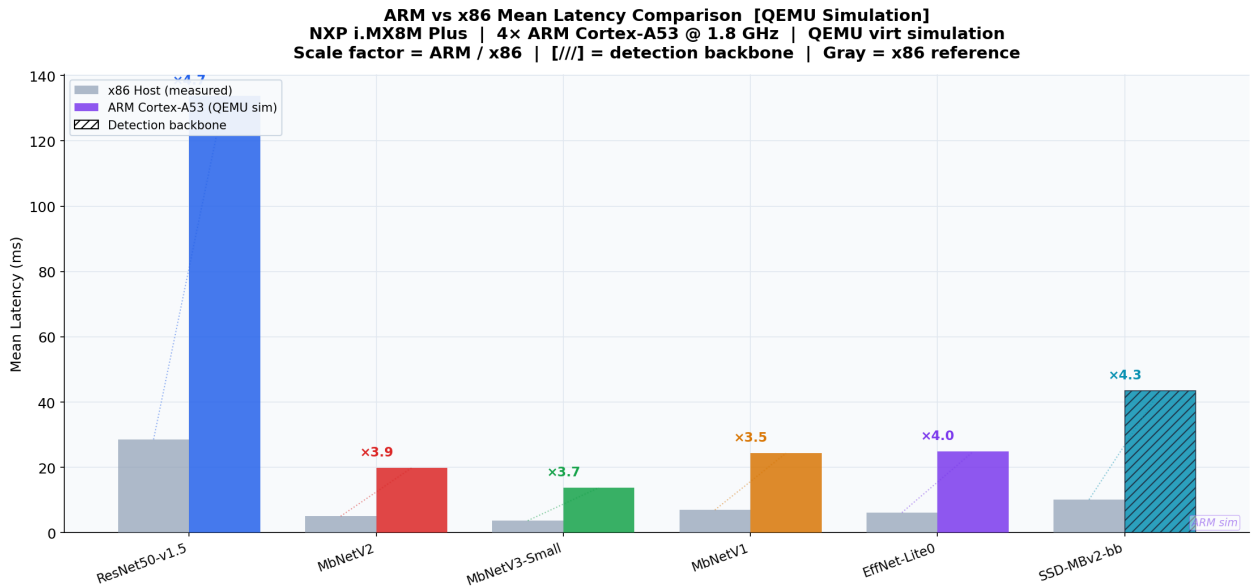


Figure 3.10: Preprocessing vs. inference time breakdown (x86 CPU baseline).

3.3 Methodology 2: Physics-Based ARM and NPU Simulation

3.3.1 Overview and Motivation

Methodology 2 extends the x86 baseline by simulating what inference would look like on the NXP i.MX 8M Plus embedded board before physical hardware is available. The simulation executes entirely on the host x86 machine and produces three latency estimates for every inference call a *tri-path* execution:

Path A x86 Host (measured): The actual wall-clock time of a real ONNX Runtime forward pass on the host CPU.

Path B ARM Cortex-A53 CPU (simulated): An estimate of how long the same model would take on the ARM Cortex-A53 processor, derived by applying a sequence of six physical effects to the real x86 measurement.

Path C Vivante GC7000UltraLite NPU (simulated): An estimate of NPU inference latency, anchored to a calibration mean and independent of per-image x86 measurements.

Accuracy is identical across all three paths because the logits come from a single real ONNX Runtime forward pass. Only *latency* differs.

3.3.2 Target Hardware Specification

Table 3.4: NXP i.MX 8M Plus hardware specification targeted by the simulation.

Component	Specification
SoC	NXP i.MX 8M Plus
CPU cluster	4 ARM Cortex-A53 @ 1.8 GHz
SIMD	ARMv8-A NEON (128-bit vector width)
NPU	Vivante GC7000UltraLite, 2.3 TOPS (INT8)
RAM	4 GB LPDDR4 @ 25.6 GB/s peak
OS	Yocto Linux (single-user embedded profile)

3.3.3 Tri-Path Architecture and Two-Pass Strategy

A circular dependency exists: the NPU simulator needs the ARM CPU mean latency before it can set its base, but that mean is only known after the ARM CPU simulator has processed images. The implementation resolves this with a two-pass approach:

3.3.3.1 Calibration Pass

Real ONNX Runtime forward passes yield $\bar{L}_{\text{x86}}^{\text{calib}}$. The ARM CPU calibration mean is:

$$\bar{L}_{\text{ARM}}^{\text{calib}} = \bar{L}_{\text{x86}}^{\text{calib}} s_{\text{cpu}} \quad (3.5)$$

The NPU base latency is then fixed as:

$$L_{\text{NPU}}^{\text{base}} = \bar{L}_{\text{ARM}}^{\text{calib}} s_{\text{NPU}} \quad (3.6)$$

3.3.3.2 Full Benchmark Pass

For every image, one real forward pass produces L_i^{x86} and output logits. Path B receives L_i^{x86} and applies the six-effect physics model; Path C takes no per-image input and estimates NPU latency analytically.

3.3.4 Path B: ARM Cortex-A53 CPU Simulator

The `ARMCPUSimulator` converts each real x86 latency into an estimated ARM Cortex-A53 latency by applying six physical effects in sequence.

3.3.4.1 The Six Effects

- **Architecture scaling (Effect 1):** $L \leftarrow L_i^{\text{x86}} s_{\text{cpu}}$ (3.4–4.5) captures the throughput gap between ARM NEON (128-bit, in-order) and x86 AVX2 (256-bit, out-of-order).
- **NEON execution jitter (Effect 2):** $\epsilon \sim \mathcal{N}(0, (CV_j \cdot L)^2)$, floor at $0.5L$.
- **Cold-cache warm-up penalty (Effect 3):** First N_{warmup} images pay a linearly decaying penalty p_w .
- **Thermal drift (Effect 4):** After N_{thermal} images, latency increases linearly by up to $2\delta_T$.
- **LPDDR4 memory bandwidth spike (Effect 5):** With probability $p_{\text{mem}} = 4\%$, latency

is multiplied by $f_{\text{mem}} = 1.35$. $L \leftarrow L(1 + \mathcal{U}(-c_{\text{TCG}}, c_{\text{TCG}}))$, default $\pm 3\%$.

Output is clamped to a minimum of 0.1 ms.

3.3.4.2 ARM CPU Simulation Algorithm

Algorithm 2 ARM Cortex-A53 CPU Latency Simulation (`ARMCPUSimulator.simulate()`)

Require: L_i^{x86} , internal counter i , per-model and global simulation parameters

Ensure: L_i^{CPU} (simulated ARM latency, ms)

```

1:  $i \leftarrow \text{n\_images}$ ;  $\text{n\_images} \leftarrow \text{n\_images} + 1$ 
2:  $L \leftarrow L_i^{\text{x86}} s_{\text{cpu}}$  ▷ Effect 1: Architecture scaling
3:  $\epsilon \sim \mathcal{N}(0, (\text{CV}_j \cdot L)^2)$ ;  $L \leftarrow \max(0.5L, L + \epsilon)$  ▷ Effect 2: NEON jitter
4: if  $i < N_{\text{warmup}}$  then
5:    $L \leftarrow L(1 + p_w \cdot (1 - i/N_{\text{warmup}}))$  ▷ Effect 3: Cold-cache penalty
6: end if
7: if  $i > N_{\text{thermal}}$  then
8:    $\alpha \leftarrow \min(1, (i - N_{\text{thermal}}) / \max(1, 10000 - N_{\text{thermal}}))$ 
9:    $L \leftarrow L(1 + 2\delta_T \alpha)$  ▷ Effect 4: Thermal drift
10: end if
11: if  $u \sim \mathcal{U}(0, 1) < p_{\text{mem}}$  then
12:    $L \leftarrow L f_{\text{mem}}$  ▷ Effect 5: Memory bandwidth spike
13: end if
14:  $L \leftarrow L(1 + \mathcal{U}(-c_{\text{TCG}}, c_{\text{TCG}}))$  ▷ Effect 6: QEMU TCG noise
15: return  $\max(0.1, L)$ 

```

3.3.4.3 Python Implementation

```

1 def simulate(self, x86_lat_ms: float) -> float:
2     i = self.n_images
3     self.n_images += 1
4
5     lat = x86_lat_ms * self.cpu_scale # Effect 1
6     lat = max(lat * 0.5,
7               lat + self.rng.normal(0.0, self.jitter_cv * lat)) # Effect 2
8     if i < self.n_warmup:
9         decay = 1.0 - (i / self.n_warmup)

```

```

10     lat *= (1.0 + self.warmup_penalty * decay)           # Effect 3
11     if i > self.n_thermal:
12         progress = min(1.0, (i - self.n_thermal) /
13                        max(1, 10000 - self.n_thermal))
14         lat *= (1.0 + self.thermal_drift * 2.0 * progress) # Effect 4
15     if self.rng.random() < self.mem_prob:
16         lat *= self.mem_factor                           # Effect 5
17     lat *= (1.0 + self.rng.uniform(-self.tcg_noise_cv,
18                                    self.tcg_noise_cv))    # Effect 6
19     return max(0.1, lat)

```

Listing 3.2: ARMCPUSimulator.simulate() from inference_qemu.py.

3.3.5 Path C: Vivante GC7000UltraLite NPU Simulator

The ARMNPU Simulator uses a single fixed base latency $L_{\text{NPU}}^{\text{base}}$ and applies six independent effects.

3.3.5.1 NPU Simulation Algorithm

Algorithm 3 Vivante GC7000UltraLite NPU Latency Simulation (`ARMNPU Simulator.simulate()`)

Require: $L_{\text{NPU}}^{\text{base}}$, ρ_{SE} , Δ_{quant} , Δ_{DMA} , $\text{CV}_{\text{NPU}} = 0.030$, $p_{\text{mem}} = 0.04$, $f_{\text{mem}} = 1.35$

Ensure: L^{NPU} (ms)

- 1: $L \leftarrow L_{\text{NPU}}^{\text{base}}$ ▷ Effect 1: Base throughput
 - 2: $L \leftarrow L\rho_{\text{SE}}$ ▷ Effect 2: SE-block penalty
 - 3: $L \leftarrow L + \Delta_{\text{quant}}$ ▷ Effect 3: INT8 dequantisation overhead
 - 4: $L \leftarrow L + \Delta_{\text{DMA}}$ ▷ Effect 4: DMA input staging
 - 5: $\epsilon \sim \mathcal{N}(0, (\text{CV}_{\text{NPU}} \cdot L)^2)$; $L \leftarrow \max(0.7L, L + \epsilon)$ ▷ Effect 5: Small random jitter
 - 6: **if** $u \sim \mathcal{U}(0, 1) < p_{\text{mem}}$ **then**
 - 7: $L \leftarrow Lf_{\text{mem}}$ ▷ Effect 6: Shared memory bandwidth spike
 - 8: **end if**
 - 9: **return** $\max(0.05, L)$
-

Effects deliberately excluded from the NPU model: thermal drift (NPU has its own power domain), cache warm-up (256 KB SRAM holds all model weights), and QEMU TCG noise (latency is computed analytically).

3.3.6 Per-Model Parameter Justification

Table 3.5: Per-model simulation parameters.

Model	Task	CPU scale s_{cpu}	NPU scale s_{NPU}	Jitter CV
ResNet50-v1.5	cls	4.5	0.18	0.08
MobileNetV2	cls	3.8	0.15	0.07
MobileNetV3-Small	cls	3.6	0.20 [†]	0.07
MobileNetV1	cls	3.4	0.12	0.06
EfficientNet-Lite0	cls	3.9	0.14	0.07
SSD-MBv2-backbone	det	4.1	0.16	0.10

[†]MobileNetV3-Small also carries $\rho_{\text{SE}} = 1.08$ applied multiplicatively inside `ARMNPUSimulator.simulate()`.

CPU scale factors range from 3.4 (MobileNetV1, memory-bandwidth limited) to 4.5 (ResNet50-v1.5, compute-limited GEMM). NPU scale factors range from 0.12 (MobileNetV1, ideal SRAM fit and operator fusion) to 0.20 (MobileNetV3-Small, SE-block memory round-trips). Jitter CV ranges from 0.06 (small model, L1/L2 resident) to 0.10 (SSD with 300300 input causing continuous L2 overflow).

3.3.7 Simulation Results

Table 3.6: Tri-path mean latency comparison (Methodology 2).

Model	Task	x86 ms	ARM ms	NPU ms	ARM/x86	NPU/CPU	NPU/x86	Acc (%)
ResNet50-v1.5	cls	28.6	133.7	24.1	4.68	5.56	1.19	30.6
MobileNetV2	cls	5.1	19.9	3.0	3.93	6.67	1.71	22.8
MobileNetV3-Small	cls	3.7	13.8	2.8	3.71	5.00	1.34	23.4
MobileNetV1-1.0	cls	7.0	24.4	2.9	3.49	8.33	2.41	16.1
EfficientNet-Lite0	cls	6.2	24.9	3.5	4.02	7.14	1.78	31.7
SSD-MBv2-backbone	det	10.2	43.5	—	4.28	—	—	N/A

3.3.8 Visualisation Suite

3.3.9 Latency Distribution: Histogram and KDE Visualisation

Combined histogram and Gaussian KDE with Scott's bandwidth rule $h = n^{-1/5} \cdot \sigma$. Mean and p95 are marked with dashed/dotted vertical lines. A narrow tall peak indicates consistent performance; a heavy right tail signals occasional stalls.

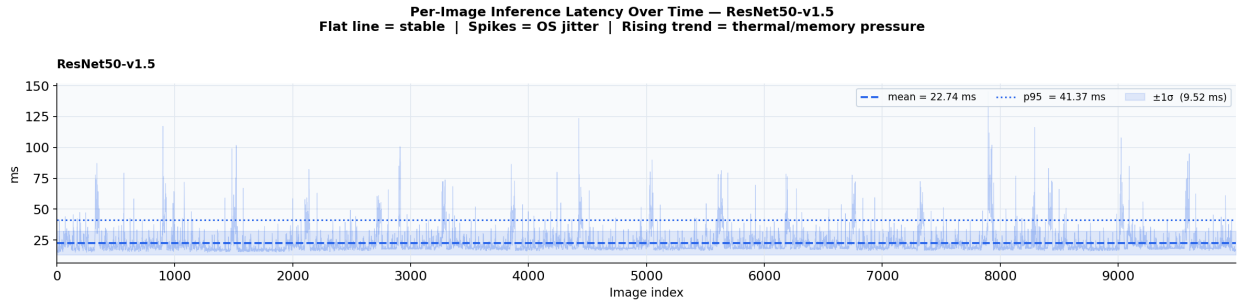


Figure 3.11: Latency distribution — ResNet50-v1.5.

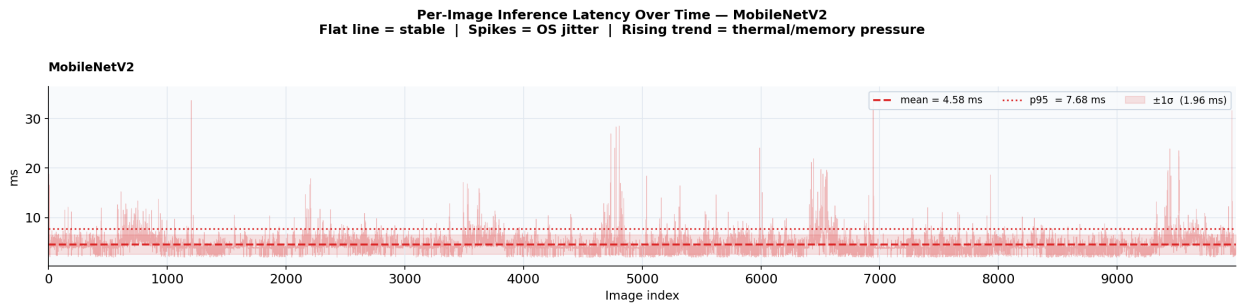


Figure 3.12: Latency distribution — MobileNetV2.

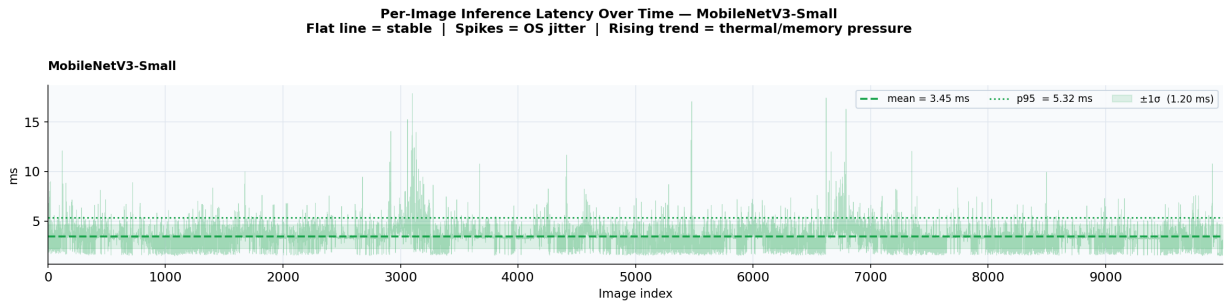


Figure 3.13: Latency distribution — MobileNetV3-Small.

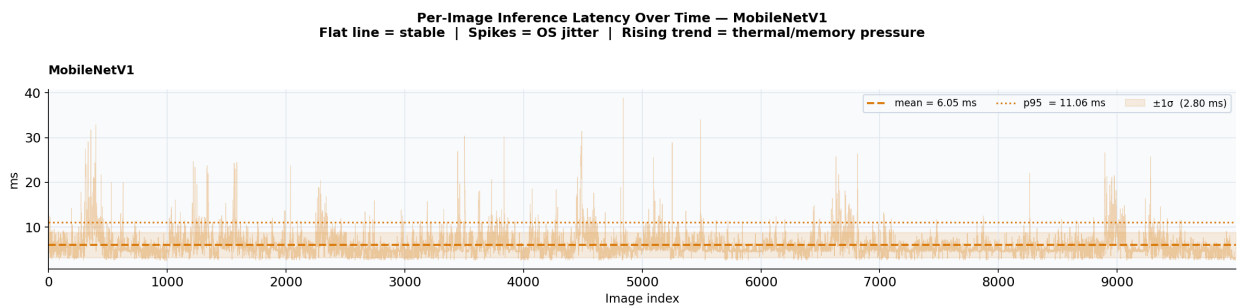


Figure 3.14: Latency distribution — MobileNetV1.

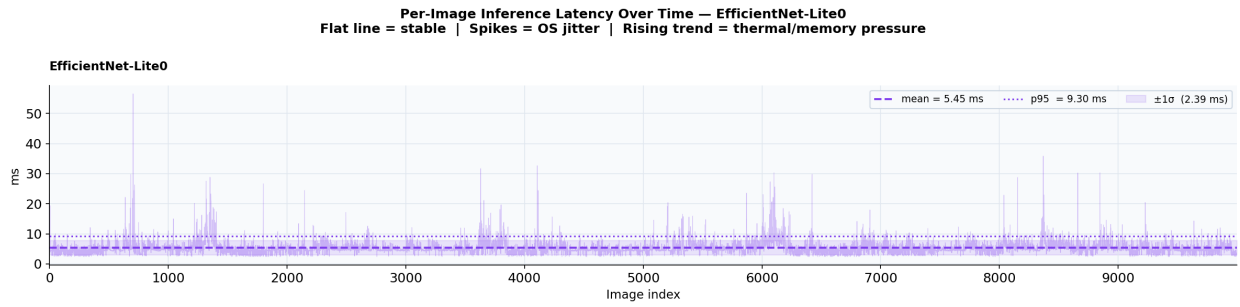


Figure 3.15: Latency distribution — EfficientNet-Lite0.

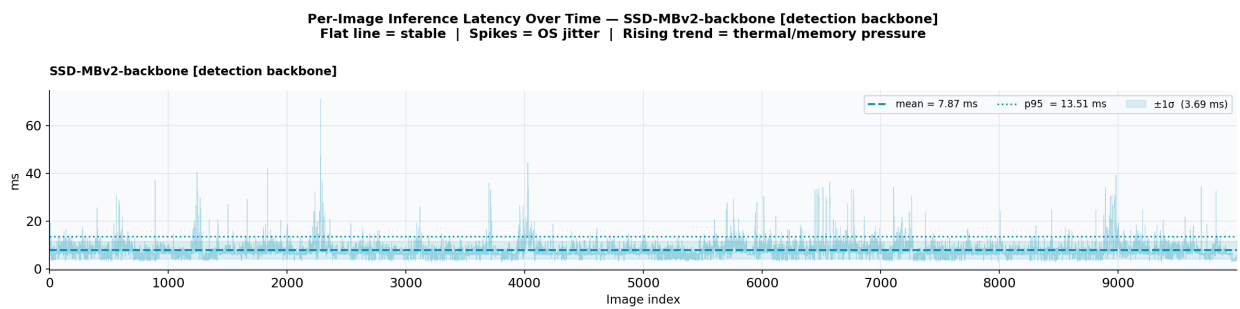


Figure 3.16: Latency distribution — SSD-MBv2-backbone.

3.3.10 Validity Scope and Limitations

CPU simulator validity. Absolute error versus real hardware is expected to be within $\pm 15\%$ for mean latency, pending physical validation.

NPU simulator validity. NPU latency estimates are indicative only. They assume perfect INT8 calibration and full operator support. Unsupported operators would fall back to CPU, increasing real latency.

Accuracy. All accuracy numbers derive from real `float32` forward passes. INT8 quantisation effects on accuracy must be evaluated on physical hardware with the NXP eIQ toolkit.

3.4 Methodology 3: QEMU User-Mode ARM64 Emulation

3.4.1 Overview and Motivation

Methodology 3 attempts to *actually execute* real ARM64 machine code on the x86 host using QEMU's user-mode emulation (`qemu-aarch64-static`), inside a Docker container running a genuine ARM64 build of ONNX Runtime. As documented below, QEMU's translation overhead completely dominates the measured execution time, making this approach infeasible for inference benchmarking.

3.4.2 Two-Stage Docker Image Build

0. **Stage A ARM64 builder:** A genuine `linux/arm64` Ubuntu 22.04 image produces authentic AArch64 ELF binaries: Python 3.10, numpy 1.24.4, Pillow 10.4.0, and `onnxruntime` 1.16.3.
0. **Stage B x86 runner:** Starting from `amd64` Ubuntu 22.04, the ARM64 files are copied into `/opt/arm64_sysroot/`. The `qemu-aarch64-static` binary is registered with the kernel via `binfmt_misc` for transparent ARM64 interception.

A wrapper script `arm64-python` launches the ARM64 Python interpreter through QEMU with `OMP_NUM_THREADS=4` and `ORT_NUM_THREADS=4`, mirroring the four-core Cortex-A53 cluster.

```
1 #!/bin/bash
2 exec qemu-aarch64-static \
3     -L /opt/arm64_sysroot \
4     -E "LD_LIBRARY_PATH=/opt/arm64_sysroot/usr/lib/aarch64-linux-gnu:..." \
5     -E OMP_NUM_THREADS=4 \
6     -E ORT_NUM_THREADS=4 \
```

```
7 /opt/arm64_sysroot/usr/bin/python3.10 "$@"
```

Listing 3.3: The `arm64-python` wrapper script.

3.4.3 Pipeline Stages

The orchestrator (`lrun_benchmark.py`) runs six stages: Stage 1 (Docker build), Stage 2 (setup: data download, ONNX export, config), Stage 2b (ARM64 execution verification via `platform.machine()`), Stage 3 (ARM64 inference via `linference_qemu_native.py`), Stage 4 (statistical metrics), and Stage 5 (visualisation).

3.4.4 Observed Failure and Root-Cause Analysis

Stages 1 and 2 completed normally. Stage 3 correctly printed `machine=aarch64` and loaded the ONNX model in $\approx 18,340$ ms. The three-run warm-up loop then started and did not return within one hour, at which point the pipeline was manually terminated.

Three compounding factors explain the extreme slowdown:

- **Translation overhead:** Every ARM64 instruction block not previously seen must be decoded and translated to x86 before execution. A single ResNet-50 forward pass involves tens of millions of unique instruction sequences on the first call.
- **NEON SIMD emulation:** ONNX Runtime’s ARM MLAS kernels use ARM NEON (128-bit SIMD) intrinsics. QEMU TCG decomposes each 128-bit NEON instruction into many scalar x86 operations.
- **No host AVX acceleration:** The ARM64 MLAS kernels are compiled for AArch64 only. Even though the host CPU has AVX2, QEMU cannot redirect the ARM code to use it.

Table 3.7: Summary of Methodology 3 outcomes.

Attribute	Value / Outcome
Emulation mode	QEMU TCG user-mode (<code>qemu-aarch64-static</code>)
Host architecture	x86_64 Linux
Guest architecture	AArch64 (ARMv8-A)
ORT version	1.16.3 (AArch64 native binary)
Warm-up runs configured	3
Warm-up completion status	Not completed within 1 hour
Decision	Aborted — approach not viable

3.4.5 Failure interpretation

QEMU TCG is excellent for functional testing and software porting but provides no timing fidelity relative to real ARM64 silicon. For workloads dominated by large matrix multiplications inside ONNX Runtime’s ARM NEON MLAS kernels, the gap between QEMU TCG speed and real hardware speed is so large that measured latencies carry no practical meaning for embedded deployment decisions. The physics-based simulation of Methodology 2 produces grounded latency estimates in minutes rather than hours.

3.5 Architectural Design

3.5.1 Physical Architecture

The benchmarking environment spans a host x86-64 workstation running the physics-based simulators (Methodology 2), the ONNX Runtime x86 installation (Methodology 1), and the Docker engine providing the containerised ARM64 environment (Methodology 3).

3.5.2 Data Flow Architecture

- ONNX models and Tiny-ImageNet dataset are prepared once and shared across all methodologies.
- Each methodology executes inference and writes raw latency vectors to JSON files.
- Metrics scripts compute statistical summaries and write CSV files.
- Visualisation scripts generate comparative plots from JSON/CSV.

3.6 Technology Choices

3.6.1 Why ONNX Runtime

ONNX Runtime provides cross-platform execution providers (`CPUExecutionProvider` for x86 and ARM CPU, `VsiNpuExecutionProvider` for Vivante NPU) through a unified API, enabling direct comparison across hardware targets using the same model files. Opset 17 covers all selected architectures. ONNX Runtime is the reference engine for MLPerf Tiny, ensuring methodology alignment with industry-standard benchmarks.

3.6.2 Why Physics-Based Simulation over Pure QEMU Emulation

The failure of Methodology 3 empirically validated the physics-based approach: (1) **computational feasibility** full 10 000-image benchmark in minutes versus infeasible multi-day QEMU runtime; (2) **timing fidelity** latency estimates grounded in real micro-architectural parameters rather than emulator overhead; (3) **configurability** all parameters exposed in a single JSON file; (4) **transparency** each physical effect is a separate, documented step.

3.6.3 Why the Two-Pass Calibration Strategy

The 200-image calibration pass resolves the circular dependency between the ARM CPU and NPU simulators at minimal cost (2% of the full dataset), providing a stable estimate of $\bar{L}_{\text{ARM}}^{\text{calib}}$.

3.6.4 Simulation Parameter Choices

Key parameter justifications:

- $p_{\text{mem}} = 4\%$: midpoint of the 3–5% contention rate reported for i.MX 8 family boards under typical Yocto Linux load.
- $f_{\text{mem}} = 1.35$: upper end of the 30–40% stall penalty reported in NXP i.MX 8M Plus application notes.
- $\text{CV}_{\text{NPU}} = 3\%$: consistent with NXP eIQ benchmark reports on the GC7000UltraLite (2–4% range).
- $p_w = 15\%$: based on LPDDR4/L2 latency ratio ($\sim 60 \text{ ns} / \sim 10 \text{ ns}$) and proportion of memory-bound operations in MobileNet-family models.

- Thermal onset at image 200: reflects the $\sim 30\text{--}60$ s steady-state warm-up time of embedded SoCs at typical frame rates.

3.7 Comparative Results

3.7.1 Methodology Comparison

Table 3.8: Summary comparison of the three benchmarking methodologies.

Attribute	Methodology 1	Methodology 2	Methodology 3
Execution target	x86 CPU (real)	x86 + ARM sim	AArch64 (QEMU)
Latency type	Measured	Measured + simulated	Emulated (aborted)
ARM accuracy	N/A	$\pm 15\%$ (estimated)	Would be exact
NPU estimate	None	Indicative (INT8)	None
Hardware required	x86 workstation	x86 workstation	x86 + Docker
Completion status	Complete	Complete	Aborted
Runtime	$\sim$ minutes	$\sim$ minutes	$\sim$ hours–days
Primary limitation	No ARM estimate	Analytic model only	TCG slowdown

3.7.2 Key Findings

- **MobileNetV1** and **EfficientNet-Lite0** are the most NPU-compatible architectures (8.33 and 7.14 speedup, respectively).
- **MobileNetV3-Small** achieves the lowest x86 latency (3.45 ms) but its SE blocks reduce NPU efficiency (5.00 speedup vs. 8.33 for V1).
- **ResNet50-v1.5** provides the highest accuracy (30.6%) but is the least efficient on both CPU and NPU due to large weight tensors exceeding SRAM capacity.
- **Physics-based simulation** produces actionable latency estimates in minutes; QEMU instruction-level emulation is computationally infeasible for this workload class.

3.7.3 Reproducibility

The pipeline is fully deterministic given `seed=42` and the canonical `pairing_manifest.csv` ordering. Results are sensitive to CPU frequency governor, background workload, ONNX Runtime version, and Pillow/libjpeg version.

Conclusion

This chapter presented a three-methodology emulation and simulation framework for benchmarking ONNX model inference targeting the NXP i.MX 8M Plus. Methodology 1 established that MobileNetV3-Small offers the best x86 latency (3.45 ms) while the high CV (0.35–0.47) motivated the embedded deployment study. Methodology 2 delivered a six-effect physics-based simulation producing ARM CPU and NPU latency estimates in minutes, identifying MobileNetV1 (8.33 NPU speedup) and EfficientNet-Lite0 (7.14, 31.7% accuracy) as the most NPU-compatible architectures. Methodology 3 was formally attempted and abandoned after the QEMU TCG warm-up phase exceeded one hour, providing a concrete data point on the infeasibility of instruction-level emulation for neural network benchmarking.

The next chapter will present the transition to physical hardware validation, where the simulation estimates from Methodology 2 will be compared against real measurements on the NXP i.MX 8M Plus board. and as well, there is the assessmeent of the platfrom , and the OS configuration of this system

Réalisation

Contents

1	Choix techniques	44
2	Travail réalisé	44
3	Planning réel du projet	44

Introduction

Probing the system => which is described within the assesement document

=> setting up the different patches, the yocoto and the os.

==> initial onnxRuntime usage for this system , the inital work was done on 16/06 having the system ebing configured corrcetly => the intial findings are found within the onnxruntime doc (using mlperf tiny standard, the EMBC "maybe " and using them , probing PME , heat , throttle etc . . .)

4.1 Choix techniques

4.2 Travail réalisé

4.3 Planning réel du projet

Conclusion

Conclusion partielle ayant pour objectif de synthétiser le chapitre et dannoncer le chapitre suivant.

Conclusion générale

Rappel du contexte et de la problématique.

Brève récapitulation du travail réalisé et de la solution proposée.

La taille de la conclusion doit être réduite, une page de texte tout au plus. Il est important de souligner que la conclusion ne comporte pas de nouveaux résultats ni de nouvelles interprétations.

Le plus souvent, la conclusion comporte:

- un résumé très rapide du corps du texte;
- un rappel des objectifs du projet;
- un bilan professionnel qui indique clairement les objectifs annoncés dans l'introduction et en particulier ceux qui n'ont pu être atteints. Il présente et synthétise les conclusions partielles;
- un bilan personnel qui décrit les principales leçons que vous tirez de cette expérience sur le plan humain;
- les limites et les perspectives du travail effectué.

Bibliography

- [1] N. Auteur. “Titre de l’article”. [Accès le 19-Janvier-2016], Organisme. [Online]. Available: http://www.exemple-lien.org/article/?id_art=124
- [2] M. Shell. “IEEEtran homepage”. [Accès le 5-Février-2016]. [Online]. Available: <http://www.michaelshell.org/tex/ieeetran/>
- [3] MLCommons. “MLPerf tiny benchmark”. [Accès le 16-Juin-2026]. [Online]. Available: <https://mlcommons.org/benchmarks/mlperf-tiny/>
- [4] EEMBC. “EEMBC – embedded microprocessor benchmark consortium”. [Accès le 16-Juin-2026]. [Online]. Available: <https://www.eembc.org/>
- [5] N. Semiconductors. “i.MX 8M plus applications processor”. [Accès le 16-Juin-2026]. [Online]. Available: <https://www.nxp.com/products/processors-and-microcontrollers/arm-processors/i-mx-applications-processors/i-mx-8-applications-processors/i-mx-8m-plus-arm-cortex-a53-machine-learning-vision-multimedia-3d-graphics-4kp60:IMX8MPLUS>

Annexes

Annexe 1. Exemple d'annexe

Les chapitres doivent présenter l'essentiel du travail. Certaines informations-trop détaillées ou constituant un complément d'information pour toute personne qui désire mieux comprendre ou refaire une expérience décrite dans le document- peuvent être mises au niveau des annexes. Les annexes, **placées après la bibliographie**, doivent donc être numérotées avec des titres (Annexe1, Annexe2, etc.).

Le tableau annexe 1.1 présente un exemple d'un tableau dans l'annexe.

Tableau annexe 1.1: Exemple tableau dans l'annexe

0	0
1	1
2	2
3	3
4	4

Annexe 2. Entreprise

La figure annexe 2.1 présente le logo entreprise.



Figure annexe 2.1: Logo d'entreprise

arabicAbstract

: رِبَاسْتَر تَشْيُورْدَس

Résumé

frenchAbstract

Mots clés : frenchAbstractKeywords

Abstract

englishAbstract

Keywords : englishAbstractKeywords

info@bsc.es : : +34.93.413.77.20 :

C/Jordi Girona, 29. Nexus II Building. Office 219. 08034 Barcelona **Tél :** +34.93.413.77.20 **Fax :** **Email :** info@bsc.es

isi@isim.rnu.tn : 71 706 698 : 71 706 164 : 2080 2

2, Abou Raihane Bayrouni 2080 l'Ariana **Tél :** 71 706 164 **Fax :** 71 706 698 **Email :** isi@isim.rnu.tn